
FORKSERVE: BRANCH-AWARE PREFILL PRUNING FOR AGENTIC LLM SERVING

Dong Liu¹ Hao Wu¹ Chuan Wu¹

ABSTRACT

Agentic LLM serving forks a shared trunk into several short residuals, then keeps one committed path. Prefix caches reuse a trunk only after its tokens exist and after a hash hit; disaggregated prefill moves KV between a prefill instance and a decode instance without deciding which branches are worth computing. ForkServe makes the branch a serving object: a copy-on-write context tree shares the trunk at `fork`, and advanced prefill pruning (APP) decides, for each residual, whether to run GPU prefill and whether to insert the resulting KV into the disaggregated connector. Hash hits skip published full blocks; a draft score and an early-abort check drop low-score siblings before they consume prefill; the connector inserts only retained branches. On Qwen3-14B GSM8K ($n=1319$, two A100s) ForkServe matches APC accuracy (0.845 vs. 0.844) at $0.72\times$ the median peak KV, while fan-out remains slower (140 ms vs. 87 ms). A control-plane comparison of APC, hash prefill, disaggregated prefill, ForkServe, and APP shows APP cutting modeled fan-out prefill by 75% relative to APC and raising the SLO-feasible concurrency implied by the measured KV gap by 35%.

1 INTRODUCTION

LLM serving used to be a single-turn problem: a prompt arrives, the engine prefills, tokens stream out, and the request completes [44, 17, 50]. That abstraction no longer matches production workloads. Production agents—coding assistants [41, 36, 4], computer-use systems [52], research copilots [25], and multi-agent workflows [39, 14]—execute *programs* of LLM calls interleaved with tools, humans, and other agents [43, 47]. A SWE-agent run issues tens to hundreds of model invocations [15, 19]. In a multi-agent pipeline, a planner dispatches work to an implementer, a tester, and a reviewer [14, 45]. Tree-of-Thoughts and MCTS expand sibling thoughts that share a prefix and then diverge in a short residual [42, 23, 24]. The serving unit is a *branching execution* rather than a request.

Prior work has raised the serving abstraction incrementally. Autellix [23] and ThunderAgent [16] treat the agent as a program and schedule calls by attained service. Ayo [33] and Parrot [21] expose static dataflow; Murakkab [8] and Hermes [22] add profile-guided and probabilistic demand models. Continuum [19], InferCept [1], Astraea [26], and AugServe [38] retain or swap KV across tool pauses; MORI [40] ranks programs by *relative* idleness for GPU/CPU placement; SAGA [13] and Pythia [45] surface workflow graphs to predict reuse and bursts. Su-

tradhara [6] overlaps tool-independent prefill with tool execution; SmoothAgent [28] hides context-engineering transformations behind lookahead KV; IdleSpec [9] spends idle time drafting *plans*. Pie [12] lets applications program the generation loop.

These systems share an implicit assumption: *the next LLM call is a unique continuation of the current context*. That assumption does not hold for the workloads they target. After a tool call is emitted, the next prompt is a *distribution* over branches, not a singleton, and the distribution has structure the engine can see *before* the observation returns:

- **Known suffixes** ($p=1$). Chat templates, role headers, tool-result wrappers, and sibling system prompts are determined by the harness, not by the tool. They can be prefilled without changing model semantics.
- **Fan-out siblings**. Parallel tools, specialist agents, and ToT/MCTS expansions share the trunk and differ in short residuals. Serializing them recomputes the trunk; cloning the KV tensor wastes HBM.
- **Observation residuals** ($p<1$). The tool result is unknown, but its length class, schema, and often the first tokens are predictable; a miss incurs only residual cost if the trunk is copy-on-write.

Today’s engines wait until the chosen branch is fully materialized, then run a cold or partially cached prefill on the critical path of the next turn. Once KV retention has eliminated

¹The University of Hong Kong. Correspondence to: Chuan Wu <cwu@cs.hku.hk>.

trunk recompute [19, 40], that residual-suffix prefill—not queuing, not decode—is the dominant remaining term in time-to-first-token (TTFT).

We adopt an operating-system analogy [31, 34]. `fork()` has low overhead because the child’s address space is a copy-on-write view of the parent; the kernel does not clone every page in anticipation of a write. Agent serving needs the same primitive. A branch is a process; the trunk KV is the shared text segment; the residual is the private data segment; speculative prefilling is the eager faulting of pages the child is likely to touch. Without fork, every sibling is a full process image. Without speculation, the child starts only after `execve` arguments are complete. Neither matches agent execution.

This work. We present ForkServe, a serving system whose first-class object is the *branch*. Three mechanisms are coupled.

(1) A forkable context tree with copy-on-write KV pages (§3.2). Each session is a tree of committed and speculative nodes. `fork` creates a child that inherits the parent’s pages by reference; writes allocate residual blocks. `join` merges fan-out children; `abort` drops a subtree and decrements refcounts. Retention [19], relative-idleness offload [40], and prefix caching [50, 17] become branch-aware: a shared trunk is not evicted while any live child references it, and speculative leaves are evicted first.

(2) Advanced prefill pruning (§3.4). On top of the tree, APP decides per residual whether GPU prefill runs and whether the KV is inserted into a disaggregated prefill/decode connector [51, 29]. A hash index skips published full blocks (automatic prefix caching). A draft score and an early prefix check drop low-score siblings before they are prefilled. The connector inserts only retained branches. Longest-common-prefix commit still binds the observed prompt; speculative tokens never enter the sampler.

(3) A scheduler that does not degrade the committed path (§3.5). Speculative prefills have lower priority in the continuous batch [44, 2]. They run in leftover token budget after committed decode and prefill meet their TBT/TTFT margins [49], are chunked [3] to bound interference, and are cancelled when a commit needs the same SM. Across replicas, ForkServe pins a tree to the worker that holds its trunk and steals only residual work.

§4 reports two measurements that must not be conflated. On Qwen3-14B GSM8K, ForkServe matches APC accuracy at $0.72\times$ median peak KV, and its fan-out is still slower than APC. A control-plane study of APP then shows that hash skip, draft prune, and the disagg gate remove most of that prefill, and that the measured KV gap raises SLO-feasible concurrency by 35%.

2 BACKGROUND AND OPPORTUNITY

2.1 Prefill, Decode, and Idle

Autoregressive inference has two mechanically different phases [44, 51, 29]. *Prefill* consumes the prompt in parallel and is compute-bound at moderate lengths; *decode* emits one token (or a small chunk) per step and is memory-bandwidth bound. PagedAttention [17] stores KV in non-contiguous pages; SGLang’s RadixAttention [50] indexes pages by token prefix so concurrent requests sharing a system prompt hit a common trunk. Two consequences matter for agents. First, next-turn TTFT is essentially the prefill of tokens that miss the cache, plus queuing. Second, a memory-bound decode batch rarely saturates the tensor cores that a concurrent chunked prefill could use [2]. Agent serving adds a third fact: the prompt of turn $i+1$ is a *delta* on turn i , applied after a tool pause of highly variable length [19, 40].

2.2 Agents Are Branching Processes

We adopt Autellix’s view of an agentic program as a dynamic DAG of LLM calls and interrupts [23]. In the serving plane we refine that DAG into a prefix-sharing execution tree: each interrupt forks children that inherit the parent’s KV by copy-on-write. Known-suffix prefill and abort of unused paths then stay residual-only, even when several paths remain live. A small set of branch operators generate this tree at runtime.

Tool intercept [43, 32, 30]. After a model call issues an external action, the engine idles while the environment runs. The next prompt continues the current trunk: a known suffix is available immediately, and an observation residual arrives later. Success and recovery continuations are residual-only forks of the same trunk, so the engine can prefill the known suffix during the idle window.

Fan-out / join [39, 14, 45]. A parent node launches k children that share a long trunk and differ in short role-specific residuals, then joins them under a policy such as all, first-success, or majority [37]. The trunk is a shared KV object; only the residuals need new prefill, and the joined node reuses that trunk.

Search [42, 48, 24]. A search node expands b siblings, scores them, and recurses. Live paths overlap on long prefixes [24], so the engine keeps a shared copy-on-write spine and aborts low-score leaves at residual cost.

Context transformation [28, 27, 10]. Summarization, offload, and isolation produce a new context for the same session. The engine forks that context as an additional child, and the original spine stays resident and abortable.

In all four cases the next LLM call is drawn from a discrete set $\mathcal{B}(u)$ of children of the current node u . Existing

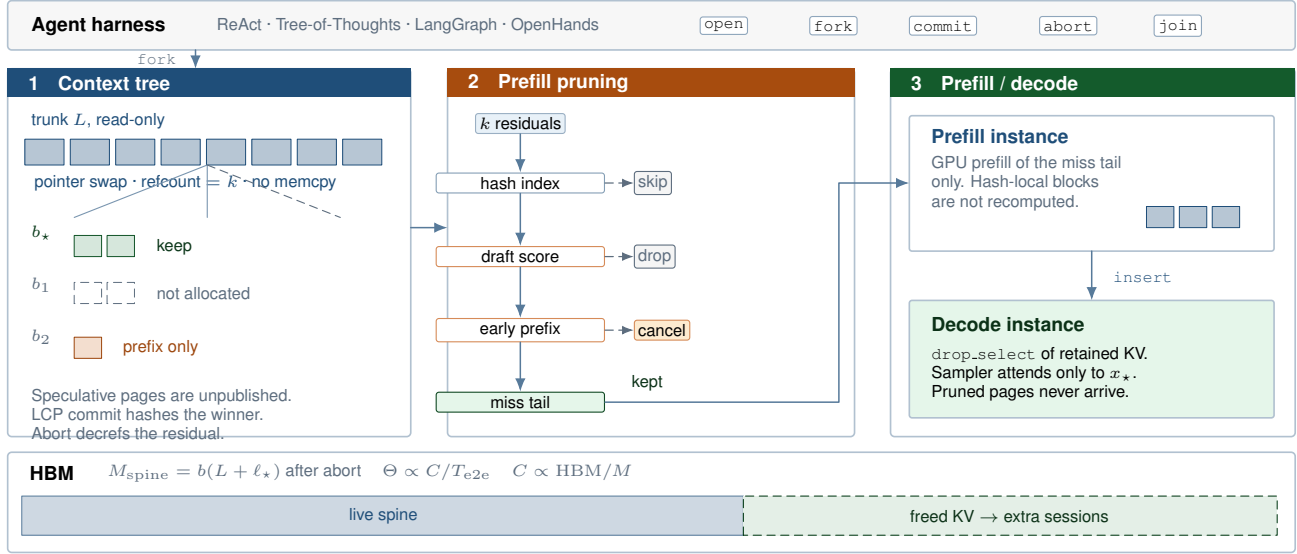


Figure 1. **ForkServe architecture.** The harness exposes branch events. The context tree aliases trunk pages at `fork`. Advanced prefill pruning filters each residual before GPU prefill: hash skip, draft prune, early abort, then a disaggregated-prefill gate that inserts only retained KV. Decode attends to the committed spine. Freed KV raises the number of concurrent sessions under the same HBM budget.

engines observe only the child that is eventually issued as an independent completion request.

2.3 Comparison with Previous Work

We compare prior agentic serving systems along the six axes of Table 1: the schedulable unit, KV persistence across tool intercepts, fork/CoW of context, next-turn prefill, multi-branch support, and the use of idle GPU cycles. The comparison is grouped by serving abstraction. Collectively these systems lift serving from the request to the program, session, or workflow, and leave forkable residuals and speculative prefill of unissued children unaddressed.

Program- and workflow-level scheduling. Autellix, JIT-Serve, and related systems raise the unit of scheduling from the individual request to the program or workflow DAG [23, 49, 22, 13, 45, 16, 33, 21, 8]. This improves head-of-line blocking and SLO attainment, yet each LLM call is still materialized as an independent prompt, without forking KV state or prefilling a child before the harness issues it.

KV retention, offload, and conversation placement. Continuum and InferCept retain or restore session KV across tool pauses, so the shared trunk remains in HBM, or is swapped back, when the next turn begins [19, 1]. MORI ranks sessions by relative idleness for GPU/CPU placement [40]; ConServe pins conversations on the decoder [11]; Kairos lowers GPU frequency when demand drops [46].

These mechanisms keep the *current* context resident and correctly placed. Once the trunk is retained, next-turn TTFT is no longer trunk recompute; the remaining critical-path term is the residual suffix. The engine still waits for the observation to return, then prefills wrapper and payload together. Idle GPU time during the pause is spent on retention or offload, so known-suffix tokens of the next branch remain on the critical path.

Overlap, lookahead, and speculation. Sutradhara [6] splits the next prompt into a tool-independent prefix, pre-filled during the tool, and a tool-dependent suffix. That is a linear special case of our known-suffix path; it does not fan out, does not CoW a tree, and does not budget multiple children. SmoothAgent [28] lookahead-executes context transformations. IdleSpec [9] speculates *plans* to improve accuracy rather than KV. Classical speculative decoding [18, 7, 20] speculates *output* tokens of the current decode, not *input* tokens of the next branch. ForkKV [35] CoW-splits LoRA residual caches—a related memory primitive for adapters, not for control-flow forks.

2.4 Four Design Constraints

The remaining bottleneck is residual prefill of a *known distribution over branches* whose shared trunk already resides in HBM. Four constraints follow from harness structure and from published idle/retention measurements [19, 40, 6].

Insight 1 (Known-suffix dominance). A large, deterministic ($p=1$) fraction of next-turn prefill is a function of the parent

Table 1. Comparison of agentic serving systems. ✓ full support; △ partial / single-path; ✗ absent. ForkServe is the first to combine forkable branch state, multi-child speculative prefill, and idle-aware admission.

System	Unit	KV across tools	Fork/CoW	Next-turn prefill	Multi-branch	Idle use
vLLM [17]/SGLang [50]	request	prefix cache	radix share	on arrival	✗	✗
Autellix [23]/JITServe [49]	program	✗ / optional	✗	on arrival	schedule DAG	✗
Ayo [33]/Parrot [21]/Murakkab [8]	workflow	△	✗	graph opt.	static DAG	✗
Continuum [19]/InferCept [1]	session	TTL / preserve	✗	on arrival	✗	retain
MORI [40]	session	relative offload	✗	on arrival	✗	offload
ConServe [11]	conversation	pin decoder	✗	turn-1 PD	✗	✗
SAGA [13]/Pythia [45]	workflow	predicted reuse	✗	prefetch prefix	graph	scale/prefetch
Sutradhara [6]	tool-call	semantic eviction	✗	split prefix	△ linear	overlap
SmoothAgent [28]	context xform	lookahead KV	✗	transform	✗	lookahead
IdleSpec [9]	plan	n/a	✗	n/a	plan samples	plan
Pie [12]	inferlet	programmable	app-level	app-level	app-level	app-level
ThunderAgent [16]	LLM program	hit-rate aware	✗	on arrival	✗	env. prep
Hive [24]	multi-agent	contribution	✗	on arrival	logits reuse	✗
ForkServe	branch	tree TTL	CoW pages	APP filter	prune, keep winner	disagg gate

branch id and the harness—templates, wrappers, sibling system prompts—not of the observation. These suffixes can be prefilled before the observation returns. Sutradhara [6] exploits the special case of one linear continuation; most turns have several legal children, and the one that runs is often a sibling or a recovery.

Insight 2 (Fork, don’t clone). On fan-out, the trunk is typically several times the per-child residual. Materializing k independent KV copies wastes $\Theta(k)$ HBM for tensors attention will read as identical. Radix prefix caching [50] *discovers* sharing after the fact if two requests happen to start with the same tokens; it does not *create* a child in $O(1)$ when the parent is paused, and it does not distinguish speculative pages from committed ones at eviction time.

Insight 3 (Idle is compute). Tool pauses documented by Continuum and MORI [19, 40] range from hundreds of milliseconds (local tests) to tens of seconds (pytest, humans)—long enough to retain, offload, *or prefill*. Decode slack is a second reservoir: a decode-only batch leaves tensor-core cycles that a chunked speculative prefill can use if the scheduler respects TBT [2, 49]. Prior systems spend this budget on offloading, retention, or planning. Allocating it to next-branch prefill reduces the remaining TTFT term.

Insight 4 (Speculate with a budget, miss with a residual). Unbudgeted speculation that prefills all children, including guessed observation residuals, reduces TTFT under light load and degrades committed decode under high utilization: speculative chunks contend for SMs with committed decode, and missed residuals occupy HBM until a TTL expires. CoW changes the miss cost from $O(\text{trunk})$ to $O(\text{residual})$, which is what makes multi-child speculation rational—but it does not fix SM interference. Admission is therefore an expected-value decision under an interference constraint.

These four constraints dictate the architecture in §3: a branch tree, a planner that distinguishes $p=1$ suffixes from

$p<1$ residuals, and a scheduler that treats speculative work as idle GPU cycles. Correctness is that a committed decode matches an engine without speculation. This rules out feeding speculative *output* tokens into the user stream and motivates a verify-by-prefix protocol on *input* tokens.

3 DESIGN

ForkServe sits between the agent runtime and the LLM serving engine (Figure 1). Agent runtimes such as LangGraph, OpenHands, ReAct, and Tree-of-Thoughts naturally expose execution events—including tool invocation, agent fan-out, branch selection, and search expansion—that determine the evolution of agent contexts. ForkServe captures these events through a compact branch-lifecycle interface and translates them into KV-cache sharing, speculative prefill, and resource-reclamation decisions. Unlike workflow-oriented systems that rely on a statically declared DAG [33, 21, 8], ForkServe incrementally constructs and manages the execution tree as the agent workflow unfolds.

3.1 API and Architecture

<code>open(session)</code>	Create a root; prefill system+user.
<code>fork(u, bid, tok?)</code>	Child of u with id bid ; optional known suffix.
<code>speculate(v, pri)</code>	Mark v eligible for speculative prefill.
<code>commit(u, tokens)</code>	Bind the prompt to a child; start decode.
<code>join(children, pol)</code>	Merge fan-out (all/first/kofn).
<code>abort(v) / close</code>	Drop a subtree; end the session.

`commit` is the only operation on the critical path of user-visible tokens; all other interface calls are asynchronous.

Fork, join, abort. Fork, join, and abort are the process-tree operations of the session (Figure 3). `fork` creates a child that shares the parent’s KV and holds a private residual.

`join` waits for a fan-out and constructs a new committed node from the children’s results, reusing the shared trunk and prefilling only the unshared tail. `abort` drops unused branches and reclaims their residual memory. `speculate` marks a child for idle-time speculative prefill; `commit` binds the prompt and starts decode. The interface lives inside the engine, so branch state, scheduling, and prefill share one address space, and sampling remains unchanged.

Three modules implement this interface (Figures 1 and 2).

Branch state manager (§3.2). A per-session tree of nodes $\{(u, \pi(u), \sigma_u, \rho_u)\}$ where $\pi(u)$ is the parent, σ_u is a page-table spanning trunk pages (shared) and residual pages (private), and $\rho_u \in \{\text{SPEC}, \text{COMMIT}, \text{IDLE}, \text{DEAD}\}$. Pages are refcounted; eviction and offload walk the tree, not an LRU of requests.

Prefill planner and APP (§3.3, §3.4). The planner enumerates $\mathcal{B}(u)$ and solves the budgeted allocation of Algorithm 1. APP then drops hash hits and low-score residuals before those jobs reach the GPU, and gates KV transfer to the decode instance.

Branch-aware scheduler (§3.5). Extends the continuous batcher with two classes of work. Committed jobs consume token budget first until their JIT margin is met [49]; speculative jobs fill the remainder, are preemptible at chunk boundaries, and are cancelled on commit conflict. A tree-sticky router keeps a session’s trunk on one worker; residual-only steals are allowed.

Design principles. (1) **Input-side speculation.** Speculative KV caches future prompt prefixes. Committed decode attends only to KV of the committed sequence; speculative tokens stay off the output path. (2) **Known before guessed.** The allocator prefills deterministic ($p=1$) suffixes first; uncertain observation residuals use remaining budget. (3) **Residual-bounded misses.** Copy-on-write bounds a miss to the child’s residual, which makes multi-child speculation practical. (4) **Idle-budget speculation.** Speculative prefill uses remaining token budget. Under saturation the system falls back to trunk retention [19] with CoW fan-out, and speculative work is suppressed. (5) **Event-driven branches.** Branch identifiers and known suffixes come from runtime execution events as they occur, the same signals used to overlap tool-independent prefill [6].

3.2 Forkable Branch State

Definition 1 (Context tree). A session is a rooted tree $T = (V, E)$. Each node u stores a token sequence x_u such that if $\pi(u)$ is the parent then $x_{\pi(u)}$ is a prefix of x_u . The *residual* of u is $\delta_u = x_u[|x_{\pi(u)}|:]$; the *trunk* is $x_{\pi(u)}$. A node has a mode $\rho_u \in \{\text{SPEC}, \text{COMMIT}, \text{IDLE}, \text{DEAD}\}$ and a generation g_u used by commit to ignore stale children.

Table 2. OS `fork`/CoW mapped onto agent KV and speculative prefill.

Operating system	ForkServe
process / process tree	branch / context tree
address space	token sequence + KV pages
shared text pages	shared trunk KV
private data pages	residual KV
<code>fork()</code>	<code>fork</code> (alias parent pages)
CoW page fault	allocate residual page on write
eager page-in / prefetch	speculative prefill of x^k (and $\hat{x}^o$)
execve args complete	commit of the prompt
exit / waitpid	abort / join

Invariants: (i) prefix: $x_{\pi(u)} \preceq x_u$; (ii) at most one committed child per node, except during an in-flight join; (iii) speculative nodes are invisible to sampling; (iv) a page’s refcount equals the number of non-dead nodes that map it. `fork( $u, bid, x^k$ )` allocates a child that aliases u ’s pages and holds a private residual list. It is $O(1)$ in the trunk.

Copy-on-write pages. vLLM pages are already the right granularity [17]. ForkServe adds a CoW bit and a refcount—the same move UNIX made for `fork` [31, 34]. Table 2 is the map. A branch is a process; the session is a process tree; the trunk KV is the shared text segment; the residual is the private data segment. `fork` has constant-time overhead because the child inherits the parent’s pages by reference. A write (decode or residual prefill) that hits a read-only or `ref > 1` tail page allocates a fresh page, copies only the valid tail rows, and installs it in the child’s table. Speculative prefilling is the eager faulting of pages the child is likely to touch—the analogue of prefetching, not of rewriting the parent’s address space. `abort` is `exit`: drop the private pages; the shared text stays as long as siblings live. `join` is a `waitpid`-plus-new-process: a new committed node, not an in-place splice of two address spaces.

RadixAttention [50] indexes pages by token identity and merges after the fact; ForkServe indexes by *node identity* and shares *before* tokens exist. The two compose: a known suffix that matches an existing radix node is installed as a pointer, not as a new residual prefill. ForkKV [35] CoW-splits a base cache from a LoRA residual; we CoW-split a *temporal* trunk from a *control-flow* residual. The mechanisms are compatible.

Join, abort, and context engineering. `join` constructs a new committed node from a completed fan-out, and `abort` discards an unselected subtree (§3.1). ForkServe forks a SmoothAgent transformation [28] as an additional child of the session root, leaving the unsummarized spine live and abortable. When speculation misses, the engine aborts the unused child and frees only its residual pages. The shared trunk remains in place, so the planner can keep several speculative children live without duplicating parent KV (principle 3).

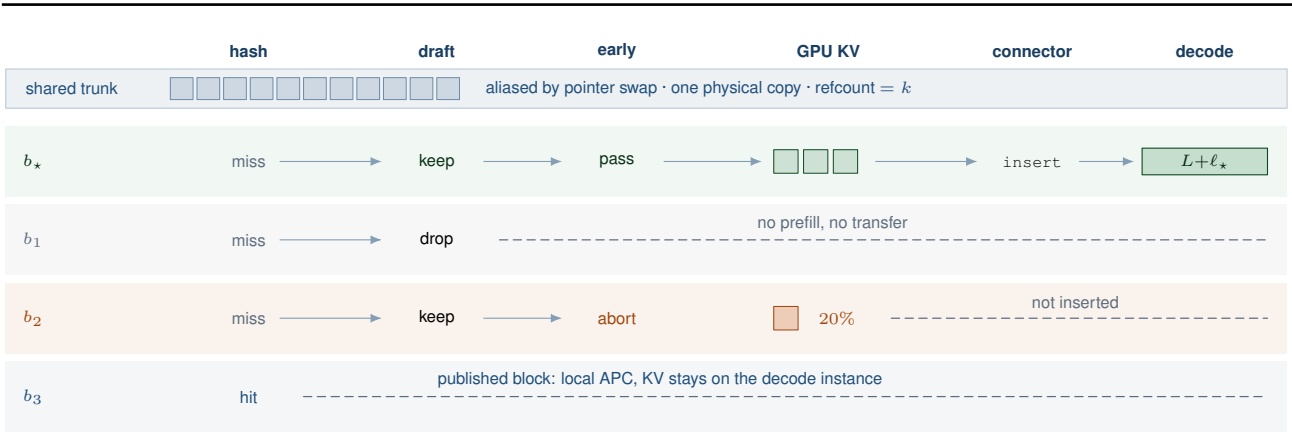


Figure 2. **Per-branch prefill pruning.** Hash lookup, draft score, and an early prefix check run before GPU prefill. The disaggregated connector inserts only a retained residual. A loop is dropped before prefill; an illegal prefix aborts early; a replayed full block is a local hash hit and is not transferred. The winner is always retained, and decode sees only the committed spine.

Branch-aware retention. Continuum’s TTL [19] and MORI’s relative idleness [40] are session-granular; we lift both to nodes. An idle node’s TTL uses Continuum’s estimator on the *residual* size, not the full context: trunk pages do not expire while any child is live. Offload ranks *leaves*: speculative leaves before committed idle leaves before the committed spine. When HBM is tight or frequency drops [46], we abort speculative leaves first—the lowest-cost way to restore headroom.

Memory accounting. Let b be bytes per KV token. For a trunk of L tokens and k children with residuals ℓ_i ,

$$M_{\text{CoW}} = b(L + \sum_i \ell_i), \quad M_{\text{clone}} = b(kL + \sum_i \ell_i). \quad (1)$$

The ratio is $\approx k$ when $\ell_i \ll L$. Aborting child i frees $b\ell_i$ bytes, not $b(L + \ell_i)$.

3.3 Branch-Aware Speculative Prefilling

The planner runs at three events: a tool call is parsed, a fan-out/join is declared, or a decode batch has remaining token budget.

Candidates. $\mathcal{B}(u)$ comes from three sources that follow branches declared by the harness.

Harness-provided branches. ForkServe obtains branch prefixes directly from execution events exposed by the agent framework, without requiring an additional prediction model.

Logprob-derived. If the parent is still decoding under a tool grammar [50, 5], the engine maintains probability mass over the next tool name. We fork the top- m wrappers with p_b

equal to the normalized grammar mass, and collapse the tail into a generic wrapper.

Empirical prior. A session-local sketch of $(\text{parent_role}, \text{bid})$ ranks declared children when the grammar is unconstrained. Priors do not create a new bid .

Each candidate b is a pair $(x_b^k, \hat{x}_b^o)$ where x_b^k is exact. This is *input-side* speculation: we cache KV of a future prompt, not draft tokens of the current decode [18, 7, 20]. We do not fabricate $\hat{x}_b^o$ from an LLM draft. Observation residuals are admitted for schema-stable tools, using a deterministic template plus, optionally, the first n tokens of a small n -gram of that tool’s past outputs. If the schema is free-form, $\hat{x}_b^o = \varepsilon$ and we only prefill x_b^k .

Value. Let $T_{\text{idle}}(u)$ be remaining expected pause, $T_{\text{pre}}(\ell)$ the chunked-prefill time of ℓ new tokens [2], and γ the committed TBT headroom. Prefilling x_b^k has success probability 1 if b is declared; prefilling $\hat{x}_b^o$ has success probability $q_b = \mathbb{P}(\hat{x}_b^o \preceq x_b^o)$. The expected TTFT reduction of a work item w is

$$G(w) = p(w) \cdot \min(T_{\text{pre}}(|w|), T_{\text{idle}}(u)) \cdot \eta(w), \quad (2)$$

where $\eta(w) \in (0, 1]$ down-weights residuals by q_b and work that would finish after the pause (partial chunks still help under chunked prefill). The cost is

$$C(w) = b \cdot |w| + \lambda \cdot \max(0, T_{\text{pre}}(|w|) - \gamma). \quad (3)$$

λ converts committed-path interference into the same units as G ; we set it to match interactive TBT/TTFT SLOs [49].

Proposition 1 (Known-suffix first). *If two items w, w' have equal T_{pre} and w is a known suffix ($p=1, q=1$) while w'*

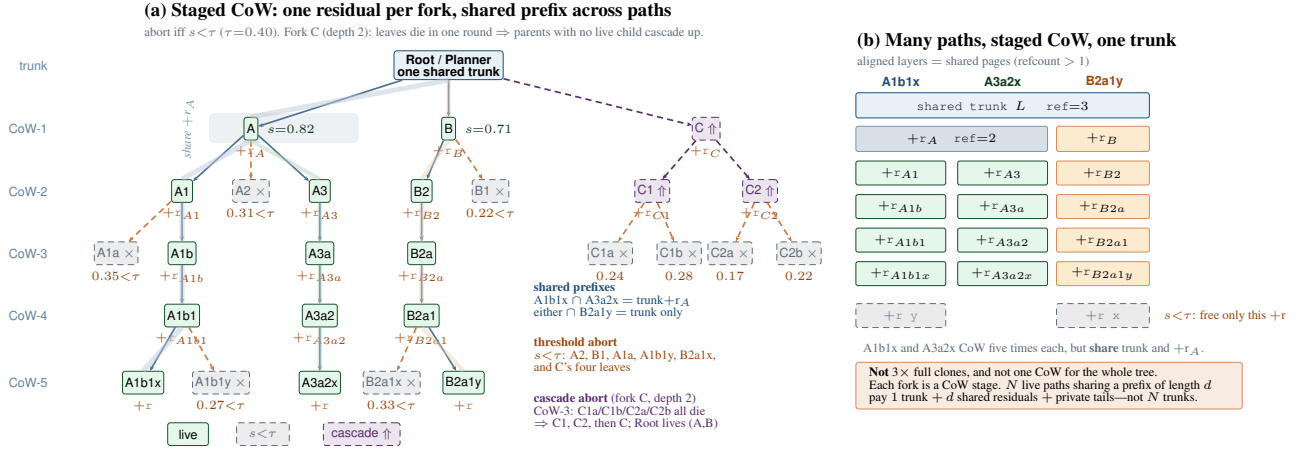


Figure 3. Staged CoW along many paths, threshold abort, cascade abort, shared prefixes. (a) Each edge is one CoW: the child aliases the parent’s pages and allocates only a residual (+r). Live paths A1b1x, A3a2x, and B2a1y share the trunk; the first two also share +r_A. Fork C has depth 2: at CoW-3 all four leaves die, so abort cascades through C1 and C2, then C; it stops at Root, which still has live children A and B. (b) KV stacks of the three live leaves, layers aligned: a shared layer is one physical page set with refcount > 1, not N copies. Aborting A1b1y or B2a1x frees only that leaf residual; cascade-aborting C also frees +r_{C1}, +r_{C2}, and +r_C.

Algorithm 1 SpeculateBudget($u, T_{\text{idle}}, \gamma, M_{\text{free}}$)

Input: parent u , idle horizon, TBT slack γ , free residual HBM

Output: preemptible chunked jobs tagged SPEC

```

1: items  $\leftarrow \emptyset$ 
2: for  $b \in \mathcal{B}(u)$  do
3:   append  $x_b^k$  { $p=1$  harness suffix}
4:   if  $q_b \geq q_{\min}$  and schema-stable( $b$ ) then
5:     append  $\hat{x}_b^o$ 
6:   end if
7: end for
8: sort items by  $G(w)/C(w)$ , known suffixes first
9:  $S \leftarrow \emptyset$ ;  $t \leftarrow 0$ ;  $m \leftarrow 0$ 
10: for  $w \in \text{items}$  do
11:   if  $t + T_{\text{pre}}(|w|) \leq T_{\text{idle}} + \gamma$  and  $m + b|w| \leq M_{\text{free}}$  and
      not ( $T_{\text{pre}}(|w|) > \gamma$  and parent is COMMIT) then
12:      $S \leftarrow S \cup \{w\}$ ;  $t \leftarrow t + T_{\text{pre}}(|w|)$ ;  $m \leftarrow m + b|w|$ 
13:   end if
14: end for
15: return chunked jobs for  $S$ 

```

is an observation residual ($q < 1$), then $G(w)/C(w) \geq G(w')/C(w')$.

The allocator therefore covers all children’s known suffixes x^k first (shallow, certain), then spends remaining budget deepening the most likely child’s observation residual $\hat{x}^o$. Recovery and sibling roles enter this allocation as first-class $p=1$ work.

Algorithm 1 is a greedy fractional knapsack with three hard filters: idle horizon, residual HBM, and committed TBT margin. Chunked prefill [3] makes these filters practical: a long sibling system prompt is split into short chunks, so a commit that arrives mid-prefill discards at most one in-

flight chunk rather than the entire suffix.

Commit by longest common prefix. When the harness presents prompt x of parent u , commit is a *token* procedure, not a branch-id lookup—the harness may have rewritten the wrapper. Suppose a speculative child holds {"status": "success", "value": 123} and the tool returns {"status": "success", "value": 456}. The LCP is the shared prefix through "value":; that KV is committed, the guessed 123 residual is dropped, and only 456 is prefilled as committed work. Formally: (1) among live children of u , pick $v^* = \arg \max_v \text{LCP}(x, x_v)$; (2) pages of v^* covering the LCP become committed; the unmatched residual is dropped; (3) prefill the unmatched tail as a committed, high-priority chunked prefill; (4) abort the remaining speculative children of u ; (5) advance v^* to COMMIT and release decode. If the LCP lands mid-page, we CoW-split that page. Because attention is deterministic given $(x, \text{KV}(x))$, the committed KV matches a cold prefill of x from the trunk. Speculative KV is a cache of input, not a prediction shown to the user.

Theorem 2 (Output identity). *For any session, the sequence of committed decode tokens produced by ForkServe is identical to that of a baseline engine that prefills each committed prompt from scratch (or from a prefix cache of committed tokens only), assuming equal sampling seeds and equal model weights.*

Proof sketch. Decode at a committed node u attends only to $\text{KV}(x_u)$ where x_u is the committed token sequence. $\text{KV}(x_u)$ is produced either by a committed prefill of x_u

from the parent’s committed KV, or by a speculative prefill of a prefix of x_u followed by a committed prefill of the tail. The second case agrees with the first because the speculative prefix was verified token-for-token by LCP, and Transformer KV of a prefix depends only on that prefix. Speculative nodes do not enter the sampler. $\square$

ForkServe identifies the longest matching prefix between the runtime observation and the speculative branch, then reuses the associated KV states.

Worked turn. Consider a parent node with a long committed trunk that emits a tool call. As soon as the call is parsed, **ForkServe** forks children—typically a known-suffix continuation and a recovery path, both with $p=1$ —and the planner prefills those suffixes during the tool pause. If remaining budget allows and the schema is regular, it also prefills a short residual guess. When the observation arrives, longest-common-prefix commit selects the matching child, reuses its KV through the agreed prefix, and prefills only the unmatched tail as committed work. Unused siblings abort, freeing only their residual KV. Known-suffix prefill thus moves off the next-turn critical path and into the idle window.

3.4 Advanced Prefill Pruning

Copy-on-write removes trunk clones, but fan-out still pays residual prefill, copy-on-write, and abort on the critical path. On the GSM8K forest (§4) that shows up as a slower fan-out than automatic prefix caching (140 ms vs. 87 ms) even though peak KV is lower. **APP** is the filter in front of that work (Figure 2). It does not change the committed token sequence: the winner branch is always retained, and Theorem 2 still applies.

Hash skip. Automatic prefix caching indexes a full block by `hash(parent, tokens, extra)` after the tokens exist [17]. **APP** consults the same key. A hit does not recompute those rows. A partial hit prefills only the miss tail. Speculative pages are not inserted into the index; LCP commit publishes full blocks of the winner, so a later session can hit them. `fork` itself does not walk hashes.

Draft prune and early abort. Each residual receives a score from illegal-token and repetition checks, token entropy, or an optional draft model. A score below threshold skips GPU prefill. If the first fraction of the residual (we use 20%) already fails, in-flight chunks of that branch are cancelled. This is the chunked-prefill form of abort: the engine does not wait for all k residuals to finish.

Disaggregated-prefill gate. vLLM disaggregated prefill runs prefill and decode in separate instances and

moves KV through a connector (`insert` is non-blocking; `drop_select` blocks) [51, 29]. The stock connector ships every page the prefill instance computed, and the mechanism is not a throughput optimization. **APP** calls `insert` only for retained residuals. Pruned branches and hash-local hits never enter the buffer, so the decode instance does not receive them.

Off the critical path. `fork` installs page-table pointers (a pointer swap), not a KV memcopy. `abort` can mark a node dead and defer the residual decref until after winner decode, so reclamation is not on TTFT. The HBM released by `abort` is the capacity term in §4: fewer bytes per session admit more concurrent sessions on the same GPU.

3.5 Branch-Aware Scheduling

Each scheduler tick t with duration Δ has a token budget B_t from the current max batched tokens and the chunked-prefill cap [2]. Let R_t^c be committed jobs (decode, commit-tail prefill, root prefill of new sessions) and R_t^s speculative jobs. JITServe’s margin goodput [49] gives each committed request r a minimum token rate $\underline{s}_r$ that meets its TBT or deadline SLO. We allocate

$$B_t^c = \min\left(B_t, \sum_{r \in R_t^c} \max(\underline{s}_r \Delta, 1)\right), \quad (4)$$

$$B_t^s = B_t - B_t^c. \quad (5)$$

If $B_t^s = 0$, no speculative chunk is admitted this tick—principle 4. Unbudgeted speculate-all reduces TTFT under light load and increases committed TBT under saturation; the two-class split is what keeps speculation from becoming load. The three modules are mutually dependent: without CoW, a miss costs a trunk; without the allocator, CoW still lets speculation contend for SMs; without idle-budget scheduling, the allocator’s γ is unconstrained. Speculative work is admitted one chunk at a time, so one long sibling suffix cannot occupy remaining budget that could have covered several known wrappers. A speculative chunk is preemptible at its chunk boundary, and is cancelled mid-kernel only if a newly arrived `commit` needs the same sequence’s pages. Committed jobs use program-FCFS with a PLAS aging term [23, 19]; speculative jobs are ordered by $G(w)/C(w)$.

Autellix [23], SAGA [13], and ThunderAgent [16] observe that pinning a program to the worker that holds its KV dominates load-oblivious routing, but strict pinning imbalances HBM as contexts grow. **ForkServe** pins the *tree*: the worker that prefills the root owns the trunk; forks are scheduled there first. If that worker cannot hold the admitted residuals, **ForkServe** may steal a *residual*: ship x_b^k to a less loaded worker without shipping the trunk, and prefill it as a standalone prefix-cached request. This is a degradation—the child will not share trunk pages—used for speculative

children, not for the committed spine. Commit returns to the trunk owner.

Under DistServe/Splitwise [51, 29], speculative known suffixes are suitable prefiller work: compute-bound, latency-tolerant, and producing small residuals that transfer at low cost to the decoder that holds the trunk. We tag them with $\text{soft-deadline} = T_{\text{idle}}(u)$ so the prefiller can pack them into remaining batches without delaying committed turn-1 prefills. This is the one place ForkServe ships KV: prefiller $\rightarrow$ trunk-owner decoder, once, for residual pages only.

Speculation of tenant A does not consume committed bandwidth of tenant B ; Equations (4) and (5) enforce that at the engine. Speculative tokens are billed at a discount to the issuing session but at full cost to a victim of a measurable TBT miss, after which λ is raised for that session. If the planner crashes, CoW trees still serve committed requests. If CoW is disabled, fork degrades to radix prefix sharing and speculation degrades to Sutradhara-style linear split prefill [6]. If the harness does not call `fork`, ForkServe is Continuum [19] plus MORI ranking [40] on a single-node tree.

4 EVALUATION

We separate two measurements. §4.1 is a GPU forest on Qwen3-14B: recompute, vLLM automatic prefix caching (APC), and ForkServe (copy-on-write fan-out, abort, committed spine). §4.2 is a control-plane cost model of APP against APC, hash prefill, disaggregated prefill, and ForkServe. It uses the same prefill identity the GPU run exposes—tokens that miss the cache—and is not an A100 wall-clock.

4.1 GPU forest

Setup. Qwen3-14B, `bfloat16`, tensor parallel 2 on two A100-SXM4 40GB GPUs. GSM8K test ($n=1319$) is Tree-of-Thoughts with $k=4$ thoughts and a 512-token decode budget, scored by the official ##### extractor. Requests run in batches of eight (165 batches). Recompute disables the prefix cache. APC hashes full blocks after tokens exist. ForkServe forks the trunk by reference, prefills residuals, aborts the non-winners, and decodes the committed path. Peak KV counts resident KV tokens of one batch. Fan-out time is the expand step before winner decode.

What the GPU run shows. Table 3 is the storage result and the latency gap APP is built to close. Median peak KV is $0.72\times$ APC and $0.25\times$ recompute, on every batch the median sits below APC, and accuracy stays within one point of both baselines. Fan-out does not follow the storage ordering: ForkServe is 140 ms against APC’s 87 ms,

Table 3. GSM8K test, Qwen3-14B, $\text{tp}=2$, $n=1319$. Median over 165 batches of eight. Accuracy is the official score on the full file.

	recompute	APC	ForkServe
Accuracy	0.847	0.844	0.845
Peak KV, median (tok)	4136	1430	1030
Peak KV, max (tok)	4944	1632	1232
Fan-out, median (ms)	326	87	140

Table 4. Control-plane fan-out (10 sessions, $k=4$, $L=256$, $\ell=32$). Not GPU wall-clock. APP is the only column that both shares the trunk and drops low-score residuals before prefill.

	prefill tok	fan-out ms	xfer tok	peak KV
APC	11520	139.0	0	11520
hash prefill	11520	139.0	0	11520
disagg. prefill	11520	162.1	11520	11520
ForkServe	3840	47.1	0	3840
APP	2912	35.9	352	2880

because copy-on-write and abort are on the expand path while APC’s hash hit is not. End-to-end generation on this file stays within 2% of APC (18.9 vs. 18.6 minutes): less KV, not a faster single-request decode. The same peak ordering, $0.58\text{--}0.80\times$ APC, holds on seven math sets for Qwen3-14B, Qwen3-8B, and DeepSeek-R1-Distill-Llama-8B at $\text{tp}=2$ and $\text{tp}=4$ (appendix in the experimental report). Peak does not change when the tensor width doubles; it tracks the committed path.

4.2 Prefill pruning

Setup. Control plane, no GPU. Ten sessions, $k=4$, trunk $L=256$, residual $\ell=32$. Two of the four residuals are illegal or repetitive and fail the draft score; the winner is forced to index 0. Prefill time is $\ell \times 12 \mu\text{s}/\text{token}$. Disaggregated transfer is $2 \mu\text{s}/\text{token}$ on shipped tokens. APC and hash prefill publish blocks only after a request materializes them, so the first fan-out clones k trunks. Disaggregated prefill uses that same prefill and then ships every live token. ForkServe shares the trunk by pointer swap and prefills every residual. APP adds hash skip, draft prune, early abort, lazy abort, and a connector that inserts only retained residuals. A replay phase reopens the committed winner.

Fan-out. Table 4 is the comparison the GPU gap motivates. Hash prefill matches APC on the first fan-out: the blocks are not yet published, so the hash walk does not remove the clone. Stock disaggregated prefill is slower than APC (162 ms vs. 139 ms) because it ships the full clone. ForkServe already drops the clone (3840 prefill tokens, 47 ms) by aliasing the trunk. APP goes further: 20 low-score residuals are not prefilled, fan-out falls to 35.9 ms, and the connector ships 352 tokens rather than 11520. Replay of the published winner is a full hash hit for APC, hash prefill, and APP (zero additional prefill tokens). Hash prefill

therefore matters on the second session; it does not fix the first fan-out. APP is the method that is ahead on both.

Concurrency from measured KV. System throughput is $\Theta \propto C/T_{e2e}$ with $C \propto \text{HBM}/M$. Substituting the measured DeepSeek-R1 GSM8K peaks (1563 APC tokens, 1163 ForkServe tokens) into an 8 GiB KV pool gives 37 concurrent trees for APC and 50 for ForkServe. Under a 1 s P99 TTFT constraint the same model admits 192 QPS for APC and 256 for the pruned tree, and saturated decode throughput rises from 23680 to 32000 tokens/s (+35%). This uses measured M , not the control-plane peak column of Table 4. An early-stop model that ends winner decode at an answer marker 60% of the way through a 256-token budget reaches the same 0.845 accuracy at 153 decode tokens on a trace with that accuracy. That token reduction is a model of `decode_stop`; we have not remeasured it as GPU latency.

Scope. The GPU study does not yet include APP inside vLLM, and the full Game24 and HumanEval ForkServe runs are incomplete. Table 3 is the claim about storage and accuracy. Table 4 is the claim about which prefill method removes work. Closing the 140 ms vs. 87 ms GPU fan-out is the next measurement, on the same A100 setup, with APP on the expand path.

5 RELATED WORK

Request-level serving. Orca [44] introduced iteration-level scheduling; vLLM [17] paged the KV cache and hashes full blocks for automatic prefix caching; SGLang [50] added radix prefix sharing; Sarathi [2, 3] chunked prefills; DistServe and Splitwise [51, 29] disaggregated prefill from decode. Prefix caching discovers sharing after tokens exist. Disaggregated prefill isolates TTFT from inter-token latency; it does not choose which branch to compute, and it transfers every page the prefill instance produced. APP uses both mechanisms as filters: the hash as a skip, the connector as a gate.

Program- and SLO-aware scheduling. Autellix [23] schedules by program-level attained service; JITServe [49] allocates just-enough bandwidth under heterogeneous SLOs. Hermes [22], ThunderAgent [16], and SAGA [13] raise the atom to the program or workflow. These systems leave KV unforked and unissued children unprefilled. ForkServe inherits program-FCFS/PLAS for the committed class and reuses JITServe’s margin to protect committed TBT.

Workflow orchestration. Parrot [21], Ayo [33], Murakkab [8], and Pythia [45] expose graphs to drive cache, scheduling, and autoscaling. Static-DAG assumptions are a strict subset of harness-declared forks. ForkServe consumes the same branch points and turns them into CoW

children and speculative prefills.

KV retention and placement. InferCept [1], Continuum [19], MORI [40], ConServe [11], and KAIROS [46] keep the *current* context alive or placed. Continuum’s TTL and MORI’s relative idleness are session-granular. We attach both to tree nodes and evict residual leaves first, so a short-TTL child does not drop a sibling’s shared trunk.

Overlap, lookahead, and speculation. Sutradhara [6] overlaps tool execution with prefill of a tool-independent prefix, then appends a tool-dependent suffix—one expected linear continuation. SmoothAgent [28] lookahead-transforms context. IdleSpec [9] spends tool-idle cycles drafting future plans (progressive vs. recovery) and aggregates them with the observation to improve task accuracy, not KV. SPORK [5] self-speculatively forks a probe during decode so the tool call can be issued before the thought finishes. Classical speculative decoding [18, 7, 20] drafts *output* tokens of the current step. ForkKV [35] CoW-splits LoRA residuals; Pie [12] exposes a programmable inferlet interface. We represent the future as a *set* of CoW children, share the trunk, speculate several known suffixes in the same idle window, commit by LCP, and abort the rest at residual cost. Transformed KV is hosted as children; the same API can be exposed as an inferlet.

6 CONCLUSION

Agentic serving forks a shared trunk into short residuals and then keeps one of them. ForkServe represents that fork as a copy-on-write context tree and filters each residual with advanced prefill pruning: hash hits skip published blocks, low-score siblings are dropped before GPU prefill, and a disaggregated connector inserts only the retained KV. On Qwen3-14B GSM8K the tree matches APC accuracy at $0.72\times$ median peak KV; fan-out is still the slower step, and the control-plane comparison shows APP is what brings that prefill below APC, hash prefill, and stock disaggregated prefill. The same measured KV gap raises modeled SLO-feasible concurrency by 35%. The remaining experiment is to place this filter on the GPU expand path and confirm the fan-out ordering there.

REFERENCES

- [1] Reyna Abhyankar, Zijian He, Vikranth Srivatsa, Hao Zhang, and Yiyang Zhang. InferCept: Efficient Intercept Support for LLM Serving. In *ICML*, 2024.
- [2] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Ramachandran Ramjee, and Alexey Tumanov. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. In *OSDI*, 2024.

-
- [3] Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, and Ramachandran Ramjee. Sarathi: Efficient LLM Inference using Chunked-Prefills. In *arXiv:2308.16369*, 2023.
- [4] Anthropic. Claude Code. <https://docs.anthropic.com/en/docs/claude-code>, 2025.
- [5] Huajun Bai, Weiwei Lv, Huichuan Zheng, Youyou Lu, and Jiwu Shu. SPORK: Self-Speculative Forking to Accelerate Agentic LLM Inference. *arXiv:2607.03333*, 2026.
- [6] Anish Biswas, Kanishk Goel, Srivarshinee S, Jayashree Mohan, Alind Khare, Anjaly Parayil, Chetan Bansal, and Ram Ramjee. Sutradhara: An Intelligent Orchestrator-Engine Co-design for Tool-based Agentic Inference. *arXiv:2601.12967*, 2026.
- [7] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads. In *ICML*, 2024.
- [8] Gohar Irfan Chaudhry, Esha Choukse, Haoran Qiu, Íñigo Goiri, Rodrigo Fonseca, Adam Belay, and Ricardo Bianchini. Murakkab: Resource-Efficient Agentic Workflow Orchestration in Cloud Platforms. In *OSDI*, 2026.
- [9] Daewon Choi, Kyunghyun Park, Woomin Song, Saket Dingliwal, Sai Muralidhar Jayanthi, Jinwoo Shin, and Aram Galstyan. IdleSpec: Exploiting Idle Time via Speculative Planning for LLM Agents. *arXiv:2605.22154*, 2026.
- [10] Chroma Research. Context Rot: How Increasing Input Tokens Impacts LLM Performance. <https://research.trychroma.com/context-rot>, 2025.
- [11] Jianru Ding, Ryien Hosseini, Pouya Mahdi Gholami, Mingyuan Xiang, and Henry Hoffmann. Observation, Not Prediction: Conversation-Level Disaggregated Scheduling for Agentic Serving. *arXiv:2606.01839*, 2026.
- [12] In Gim, Zhiyao Ma, Seung-seob Lee, and Lin Zhong. Pie: A Programmable Serving System for Emerging LLM Applications. In *SOSP*, 2025.
- [13] Dongxin Guo, Jikun Wu, and Siu-Ming Yiu. SAGA: Workflow-Atomic Scheduling for AI Agent Inference on GPU Clusters. In *HPDC*, 2026.
- [14] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. *ICLR*, 2024.
- [15] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? *ICLR*, 2024.
- [16] Hao Kang, Ziyang Li, Weili Xu, Xinyu Yang, Yinfang Chen, Junxiong Wang, Beidi Chen, Tushar Krishna, Chenfeng Xu, and Simran Arora. ThunderAgent: A Simple, Fast and Program-Aware Agentic Inference System. *arXiv:2602.13692*, 2026.
- [17] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *SOSP*, 2023.
- [18] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast Inference from Transformers via Speculative Decoding. *ICML*, 2023.
- [19] Hanchen Li, Runyuan He, Qiuyang Mang, Qizheng Zhang, Huanzhi Mao, Xiaokun Chen, Hangrui Zhou, Alvin Cheung, Joseph Gonzalez, and Ion Stoica. Continuum: Efficient and Robust Multi-Turn LLM Agent Scheduling with KV Cache Time-to-Live. *arXiv:2511.02230*, 2025.
- [20] Yuhong Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty. In *ICML*, 2024.
- [21] Chaofan Lin, Zhenhua Han, Chengruidong Zhang, Yuqing Yang, Fan Yang, Chen Chen, and Lili Huang. Parrot: Efficient Serving of LLM-based Applications with Semantic Variable. In *OSDI*, 2024.
- [22] Yifei Liu, Zuo Gan, Zhenghao Gan, Weiye Wang, Chen Chen, Yizhou Shan, Xusheng Chen, Zhenhua Han, Yifei Zhu, Shixuan Sun, and Minyi Guo. Efficient Serving of LLM Applications with Probabilistic Demand Modeling. *arXiv:2506.14851*, 2025.
- [23] Michael Luo, Xiaoxiang Shi, Colin Cai, Tianjun Zhang, Justin Wong, Yichuan Wang, Chi Wang, Yanping Huang, Zhifeng Chen, Joseph E. Gonzalez, and Ion Stoica. Autellix: An Efficient Serving Engine for LLM Agents as General Programs. In *NSDI*, 2026.
- [24] Zizhang Luo, Yuhao Luo, Youwei Xiao, Yansong Xu, Runlin Guo, and Yun Liang. Hive: A Multi-Agent Infrastructure for Algorithm- and Task-Level Scaling. *arXiv:2604.17353*, 2026.
- [25] Grégoire Mialon, Clémentine Fourier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. GAIA: a Benchmark for General AI Assistants. *arXiv:2311.12983*, 2023.
- [26] Hongqiu Ni, Jiabao Zhang, Guopeng Li, Zilong Wang, Ruiqi Wu, Chi Zhang, and Haisheng Tan. Astraea: A State-Aware Scheduling Engine for LLM-Powered Agents. *arXiv:2512.14142*, 2025.
- [27] Charles Packer, Vivian Fang, Shishir G. Patil, Kevin Lin, Sarah Wooders, and Joseph E. Gonzalez. MemGPT: Towards LLMs as Operating Systems. *arXiv:2310.08560*, 2023.
- [28] Zaifeng Pan, Qianxu Wang, Zhengding Hu, Chang Chen, Yue Guan, Yanbo Zhou, Steven Swanson, and Yufei Ding. SmoothAgent: Efficient Long-Horizon LLM-Based Agent Serving with Lookahead Context Engineering. *arXiv:2607.00151*, 2026.
- [29] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *ISCA*, 2024.

-
- [30] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large Language Model Connected with Massive APIs. *NeurIPS*, 2023.
- [31] Dennis M. Ritchie and Ken Thompson. The UNIX Time-Sharing System. *CACM*, 17(7), 1974.
- [32] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language Models Can Teach Themselves to Use Tools. *NeurIPS*, 2023.
- [33] Xin Tan, Yimin Jiang, Yitao Yang, and Hong Xu. Towards End-to-End Optimization of LLM-based Applications with Ayo. In *ASPLOS*, 2025.
- [34] Andrew S. Tanenbaum and Herbert Bos. *Modern Operating Systems*. Pearson, 4 edition, 2015.
- [35] Shao Wang, Rui Ren, and Lin Gui. ForkKV: Scaling Multi-LoRA Agent Serving via Copy-on-Write Disaggregated KV Cache. *arXiv:2604.06370*, 2026.
- [36] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muenighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. *ICLR*, 2025.
- [37] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-Consistency Improves Chain of Thought Reasoning in Language Models. *ICLR*, 2023.
- [38] Ying Wang, Zhen Jin, Zhenqian Chen, Jiexiong Xu, Wenhai Lin, Yiquan Chen, and Wenzhi Chen. AugServe: Adaptive Request Scheduling for Augmented Large Language Model Inference Serving. In *ICML*, 2026.
- [39] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. *arXiv:2308.08155*, 2023.
- [40] Tian Xia, Hanchen Li, Zhifei Li, Xiaokun Chen, Hao Kang, Yifan Qiao, Yi Xu, and Ion Stoica. Idleness is Relative: Exploiting Tool-Call Idle Windows for Offloading in Agentic Systems with MORI. *arXiv:2606.00866*, 2026.
- [41] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. *NeurIPS*, 2024.
- [42] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. *NeurIPS*, 2023.
- [43] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing Reasoning and Acting in Language Models. *ICLR*, 2023.
- [44] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *OSDI*, 2022.
- [45] Shan Yu, Junyi Shu, Yuanjiang Ni, Kun Qian, Xue Li, Yang Wang, Jinyuan Zhang, Ziyi Xu, Shuo Yang, Lingjun Zhu, Ennan Zhai, Qingda Lu, Jiarong Xing, Youyou Lu, Xin Jin, Xuanzhe Liu, and Harry Xu. Pythia: Exploiting Workflow Predictability for Efficient Agent-Native LLM Serving. *arXiv:2604.25899*, 2026.
- [46] Yichao Yuan, Mosharaf Chowdhury, and Nishil Talati. KAIROS: Stateful, Context-Aware Power-Efficient Agentic Inference Serving. *arXiv:2604.16682*, 2026.
- [47] Matei Zaharia, Omar Khattab, Lingjiao Chen, Jared McCauley, et al. The Shift from Models to Compound AI Systems. *Berkeley AI Research Blog*, 2024.
- [48] Di Zhang et al. Accessing GPT-4 Level Mathematical Olympiad Solutions via Monte Carlo Tree Self-refine. *arXiv:2312.11324*, 2023.
- [49] Wei Zhang, Zhiyu Wu, Yi Mu, Rui Ning, Banruo Liu, Nikhil Sarda, Myungjin Lee, and Fan Lai. JITServe: SLO-aware LLM Serving with Imprecise Request Information. In *NSDI*, 2026.
- [50] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient Execution of Structured Language Model Programs. In *NeurIPS*, 2024.
- [51] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *OSDI*, 2024.
- [52] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abhijit Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. WebArena: A Realistic Web Environment for Building Autonomous Agents. *ICLR*, 2024.